

Contents

legoESM Ocean Model — Design Dossier	1
Executive summary	1
1. Continuous equations	4
2. Vertical coordinate	6
3. Horizontal grids	7
4. Staggering choice	8
5. Discrete operators per grid	9
6. Coriolis and PV-flux discretization	11
7. Pressure gradient force	12
8. Equation of state	14
9. Tracer advection	15
10. Momentum advection	17
11. Time integration of the slow (3-D) modes	18
12. Barotropic–baroclinic mode splitting	20
13. Horizontal dissipation	22
14. Vertical mixing	23
15. Mesoscale eddy parameterization (GM/Redi)	25
16. Bottom drag	26
17. Surface forcing slots (ocean side only)	27
18. Bathymetry and land treatment	29
19. Halo exchange and parallelism	30
20. Differentiability and JAX considerations	31
21. What we run in production	32
22. Open scientific questions for Alistair	32
Bibliography	35

legoESM Ocean Model — Design Dossier

A working document for discussion with Alistair Adcroft (GFDL/Princeton). Compiled from the source tree as of branch `main` at commit `d771d19b` (May 2026), cross-checked against project memory and the design notes under `docs/ocean/` and `docs/ocean_experiments/`.

Audience: senior ocean dycore developer. All textbook material is suppressed. Citations are anchored in the source files and reference list at the end. Where the model is broken or compromised, this is called out explicitly under the “Known issues / caveats” line of each section.

Executive summary

legoESM is a fully differentiable Earth-system model written in pure JAX. Its ocean component is being developed by adapting standard production-ocean numerics into the JAX/pytree/`lax.scan` programming model, with `jax.grad` end-to-end as a *design constraint* — not an add-on. The dycore is hydrostatic, Boussinesq, free-surface, in a generalised vertical coordinate (z^* with partial bottom cells), and is expressed in vector-invariant form on the C-grid.

Continuous formulation

- **Boussinesq** with $\rho_0 = 1025 \text{ kg m}^{-3}$ (`legoesm.constants.rho_ocean`, re-exported as `ocean.eos.rho_0`).
- **Hydrostatic**, primitive equations.
- **Free surface** with a single prognostic SSH $\eta(\mathbf{x}, t)$ and an under-the-hood split between fast (2-D depth-integrated) and slow (3-D baroclinic) modes.
- **Vector-invariant** momentum: KE-gradient, PV flux, vertical advection, dissipation; planetary Coriolis applied separately as a forward-backward (Matsuno) update on both grids.
- **Thickness-weighted** flux-form tracer transport. Tracer conservation is *structural* — there is no “rescale T by $V_{\text{old}}/V_{\text{new}}$ ” fixer in the default path (`use_conservation_fixer = False`).

Vertical coordinate

- **z^*** with optional **partial bottom cells** (`ocean.vertical.OceanZStarCoordinate`, `OceanPartialCellCoordinate`); the latter follows [Adcroft, Hill & Marshall 1997] and [Adcroft & Campin 2004].
- The Jacobian $J = (\eta + H)/H_{\text{max}}$ is dynamic (recomputed each timestep).

Horizontal grids supported

Grid	Type	Status
Lat-lon C-grid (uniform or Mercator)	Production	Primary global path. AMIP.
MPAS Voronoi (global ico, regional, channel)	Production	Validated 5-yr at ico-4 with implicit-CN barotropic.
Cubed-sphere C-D grid	Stale / unstable	Face-boundary exponential instability; <i>excluded</i> from global wind matrix.
Spectral (Gaussian)	Diagnostic	Standalone, not in the main matrix.
SFNO (data-driven)	Experimental	Neural surrogate; not a dycore.

All structured-mesh grids use **pure Arakawa C-grid** staggering for the ocean. The cubed-sphere ocean was kept on C-D for legacy reasons but is being deprecated in favour of a pure-C-grid LLC-style design (memory: `project_cgrid_cubesphere_ocean.md`).

Time integration

- **Split-explicit** baroclinic + barotropic, with the barotropic mode optionally **fully implicit** (single-step Crank-Nicolson + PCG Helmholtz) on the lat-lon and MPAS paths. The implicit-CN path is the production default on MPAS and the preferred path on lat-lon when the C-grid Coriolis null mode contaminates diagnostics.

- Outer baroclinic step: **forward Euler** for momentum + forward-backward Coriolis, with optional **AB2** or **SSP-RK3** for the tracer-advection sub-step. SSP-RK3 / SSP34 / SSP54 / RK4 are also available as generic time integrators (`legoesm.timestepping.dispatch`) and are used by the spectral ocean.
- The 3-D step is run inside `jax.jit`, and the full segment loop inside `jax.lax.scan` for AD-safe trajectories (`build_segment_fn(...).raw` pattern).

Dissipation menu

- Constant Laplacian A_h , biharmonic B_h (on the *baroclinic perturbation* $\mathbf{u}'$ only — depth-integral $\equiv 0$).
- Biharmonic Smagorinsky C_{smag} and Laplacian Smagorinsky $C_{\text{smag_lap}}$ (lat-lon and MPAS).
- Leith and modified Leith C_L (lat-lon and MPAS).
- Biharmonic on relative vorticity $K_{\zeta, \text{bih}}$ (MPAS) for the ζ -checkerboard null mode of the energy-conserving PV flux.
- **Barotropic-mode** Laplacian (`barotropic_u_viscosity`) and biharmonic (`B_h_barotropic`, `barotropic_u_biharmonic`) targeted at the depth-mean velocity only.
- APVM (Sadourny–Basdevant; Ringler et al. 2010) on the PV at vertices (MPAS).
- 2-D Laplacian damping on the barotropic substep (`barotropic_diffusion_alpha`) and **divergence damping** (`barotropic_div_damp`, `div_damp_2`, `div_damp_4`).
- Equatorial-Gaussian boost on lateral viscosity (`equatorial_visc_boost`, `A_h_eq_boost`) — needed to control an equator-trapped mode at $f \rightarrow 0$ under the implicit-CN solver (see memory: `project_mpas_etopo_instability.md`).

Parameterizations available

- **EOS**: Wright 1997 (default; MOM6 form) and configurable linear.
- **GM/Redi**: lat-lon C-grid has both *centered* and *Griffies (1998) triad* slope schemes; MPAS has *centered only* (triads = Phase 5 of the GM/Redi MPAS plan).
- **KPP** (LMD94) + Pacanowski–Philander Richardson + constant background; convective adjustment via enhanced diffusion or entraining plume.
- **Linear / quadratic bottom drag**, with optional Killworth–Edwards / MOM6 BBL_thick_min distributed BBL formulation.
- **Shortwave penetration** (Paulson & Simpson 1977 / Jerlov water types).
- **Tracer advection menu**: upwind, TVD (Van Leer), PPM, PPM+FCT (Zalesak), DST-3 (MITgcm Scheme 33), DST-3 multidim, WENO-5/7 (Silvestri et al. 2024), SOM (Prather 1986).
- **Momentum advection menu**: vector-invariant (centered KE, or Hollingsworth–Kållberg KE), WENO-5/7 momentum advection (Silvestri 2024).
- **Surface forcing**: prescribed, restoring (with arbitrary 2-D targets, Q_{sr} split, implicit-Euler integration), combined, bulk-formulas, none.

Production defaults

The ocean test matrix at `scripts/ocean_test_matrix/testcase.py` exercises 100+ runs spanning rest-state preservation, barotropic gravity wave, wind-driven gyres (regional and global, barotropic and baroclinic), geostrophic adjustment, Phillips two-layer, inertia-gravity wave, lock exchange, overflow, Stommel gyre tracer, Eady baroclinic instability, ACC channel.

Production AMIP runs and global overturning experiments use: - lat-lon C-grid OR MPAS ico-4; - z^* + partial cells; - Wright EOS; - vector-invariant momentum with Hollingsworth KE (lat-lon); - TVD or PPM tracer advection; - KPP + enhanced-diffusion convection; - GM/Redi with triads (lat-lon) or centered (MPAS); - implicit-CN barotropic; - linear bottom drag with BBL distribution; - forward Euler + forward-backward Coriolis baroclinic stepping; - AB2 tracer time integration where applicable.

Open scientific questions for Alistair (full list in section 22)

1. Cubed-sphere face-boundary exponential instability — has Alistair seen the analogue in MITgcm cubed-sphere ocean experiments?
2. Lat-lon barotropic mode noise — best practice replacement for our cosine time filter?
3. MPAS+ETOPO bottom-trapped instability at multi-year — algorithm limit or fixable?
4. GM/Redi triad on MPAS Voronoi — does it exist anywhere correctly?
5. Should we move to fully implicit barotropic everywhere?
6. Time filter design — cosine vs MOM6 boxcar with predictor.
7. Vertical coordinate — should legoESM adopt MOM6's ALE Lagrangian
 - remap, or is z^* + partial cells sufficient for differentiable workflows?
8. Tracer transport in vanishing layers — what does MOM6 do?

1. Continuous equations

What is configurable in legoESM today

The continuous formulation is essentially fixed at the framework level. The only knob is the **equation of state** (Wright 1997 vs linear; `OceanConfig.eos`). Free-surface treatment, Boussinesq vs non-Boussinesq, hydrostatic vs non-hydrostatic are not configurable.

What we currently default to / use in production

The 3-D primitive equations are integrated in **flux form for tracers** and **vector-invariant form for momentum**, on a free-surface, hydrostatic, Boussinesq base, with reference density $\rho_0 = 1025 \text{ kg m}^{-3}$ (`constants.rho_ocean`, re-exported from `ocean.eos.rho_0`). The thickness-weighted tracer form

$$\frac{\partial(h_k T)}{\partial t} = -\nabla \cdot (h_k \mathbf{u}_k T) - \frac{\partial}{\partial z}(w T) + h_k \mathcal{F}_T,$$

is enforced by construction in all production stepper paths; see `ocean.dynamics.ocean_model_latlon_cgrid:step` (lines 798–1050) and `ocean.dynamics.ocean_model_mpas:step` (lines 607–700). The momentum tendency on the *baroclinic perturbation* $\mathbf{u}' = \mathbf{u} - \bar{\mathbf{u}}$ is

$$\frac{\partial \mathbf{u}'}{\partial t} = -q_\zeta \mathbf{F}_\perp - \nabla \left(\frac{1}{2} |\mathbf{u}|^2 + p' / \rho_0 \right) - w \partial_z \mathbf{u}' + \mathcal{D}(\mathbf{u}') + \mathcal{F}_u,$$

with $q_\zeta = \zeta/h$ (relative-only) and the planetary Coriolis $f\hat{\mathbf{z}} \times \mathbf{u}$ deferred to a separate forward-backward update. The split is documented at the top of `ocean_pe_mpas.py` (lines 13–28) and is needed to close the MPAS depth-mean Coriolis double-counting (issue #103; memory `project_global_barotropic_wind.md`).

Brief rationale

Boussinesq is the production-OGCM consensus ([Adcroft et al. 2019], [Madec et al. 2022], [Ringler et al. 2013]). Hydrostatic is sufficient at the resolutions we target (~100 km down to ~10 km regional). Free surface is needed for tidal/coupled operation. Vector-invariant carries the natural decomposition into $(\zeta + f) \times \mathbf{u}$, ∇KE and $-\nabla p / \rho_0$ that maps cleanly onto C-grid stencils ([Sadourny 1975], [Arakawa & Lamb 1977]).

The split of planetary Coriolis out of the PV flux (using $q = \zeta/h$ in the energy/enstrophy-conserving operators, then adding $f \times \mathbf{u}$ separately by a forward-backward Matsuno step) is a known trade-off ([Ringler et al. 2010] §6; memory: `project_global_barotropic_wind.md`). It guarantees stable depth-mean stepping but breaks the Ringler–Thuburn–Skamarock–Klemp energy-conservation identity that holds *only* for the full PV. We treat this as scheduled debt (issue #160).

Literature pointer

[Griffies 2004], [Griffies & Adcroft 2008], [Adcroft et al. 2019].

Known issues / caveats

- Non-Boussinesq is not implemented; we cannot quantify steric height effects without further work ([Losch, Adcroft & Campin 2004] estimate ~0.5 % in steric SSH).
- Nonhydrostatic is not planned. Resolution targets remain hydrostatic-safe.
- We are missing a clean mode-splitting *flux-reconciliation* layer in the formal sense of MOM6’s continuity-equation module. The current implementation uses Hallberg–Adcroft 2009-style barotropic correction in the tracer mass flux (`u_corrected = u_3d + (\text{Hu}_{\text{avg}} - \text{Hu}_{3D})/H_u`; `ocean_model_latlon_cgrid.py:864–880`, `ocean_model`). It is conservative; it is not identical to MOM6’s reconciled-flux design — this is a question for Alistair.

2. Vertical coordinate

What is configurable in legoESM today

Coord type	Class	Where
z^* (legacy)	<code>OceanZStarCoordinate</code>	<code>ocean/vertical.py:25</code>
z^* + partial cells	<code>OceanPartialCellCoordinate</code>	<code>ocean/vertical.py:139</code>

Construction helpers: `create_ocean_z_star()` (stretched hyperbolic tangent profile from `dz_surface` to `dz_deep` totaling `H_max`), `create_partial_cell_coordinate(z_coord, H_bathy)` (folds per-column bottom partial thickness into `h_partial`, `bottom_level`, `is_active`).

What we currently default to / use in production

- z^* + partial cells everywhere realistic bathymetry is in play.
- Pure z^* for idealised flat-bottom and channel tests.
- ALE (Adcroft & Hallberg 2006) Lagrangian-vertical step is **not** implemented. The model is *Eulerian z^* -style*: we do not have a generic Lagrangian/remap stage.

Brief rationale

z^* is the de-facto contemporary default in MOM6 and MPAS-O — it avoids vanishing surface layers under deep depressions of η , gracefully handles wetting-of-near-coastline, and admits a simple free-surface formulation. Partial cells follow [Adcroft, Hill & Marshall 1997] and are the standard fix for staircase form-drag and PGF noise on real bathymetry. We chose this progression over σ /isopycnal/ALE because partial cells compose with z^* and require no rework of existing operators (see `docs/ocean_experiments/partial_cells_plan.md`).

Literature pointer

[Adcroft & Campin 2004]; [Adcroft, Hill & Marshall 1997]; [Adcroft & Hallberg 2006].

Known issues / caveats

- The partial-cell PGF is the *seed* of a bottom-trapped instability that is the principal scientific concern at multi-year runs (see §13 and memory `project_mpas_etopo_instability.md`).
- Partial cells are differentiable only piecewise — `bottom_level` is integer-valued. AD is smooth in H_{bathy} as long as the bottom level index doesn’t change. (`docs/ocean_experiments/partial_cells_plan`. “Differentiability Contract”).
- We have not implemented ALE remap. A *case for* ALE comes from spurious diapycnal mixing in z -coordinates ([Griffies, Pacanowski & Hallberg 2000]; [Megann 2018]). A *case against* is differentiability: the remap step is piecewise and has discontinuities when layer ordering changes ([White & Adcroft 2008]).

3. Horizontal grids

What is configurable in legoESM today

Grid family	Constructor	Storage layout	Status
Lat-lon C-grid (uniform)	<code>grids.latlon.create_latlon_grid</code>	scalars at cells; u at lon faces (n_lat, n_lon+1); v at lat faces ($n_lat+1,$ n_lon)	Production
Lat-lon C-grid Mercator (isotropic)	<code>grids.latlon.create_mercator_grid</code>	same	Production
Lat-lon regional / channel	reuse via mask	same	Production
MPAS Voronoi global ico-3..6	<code>grids.voronoi</code>	scalars at cells; $u_{\perp}$ at edges	Production
MPAS Voronoi regional	same factory, with mask	same	Production
MPAS Voronoi periodic channel	same factory, <code>periodic_L_rad</code>	same	Production (Eady, ACC)
Cubed-sphere C-D grid	<code>grids.cubed_sphere_cdgrid</code>	scalars at face centers ($6, n, n$); D-grid winds at corners	Stale, instability
Spectral (Gaussian)	<code>grids.gaussian.GaussianGrid</code>	Grid coefficients per level	Diagnostic only

The lat-lon C-grid is the canonical case and serves as the reference for operator semantics. The MPAS path uses **TRISK** ([Thuburn et al. 2009]; [Ringler et al. 2010]).

Bathymetry and land-mask atomicity

- Land mask, u -face mask, v -face mask are stored as separate fields of `LatLonCGridOceanState` (`ocean/state.py:281`).
- They must be **mutually consistent**: a face is wet only if both flanking cells are wet. `compute_face_masks(land_mask)` is the canonical builder.
- A common bug is to replace `land_mask` post-hoc via `state._replace(land_mask=...)` and leave `u_mask`, `v_mask` stale — this allows mass flux through walls. The fix is the helper `replace_land_mask(state, new_mask)` in `ocean/init_latlon_cgrid.py:306` that atomically rebuilds all three. A runtime check (`enable_runtime_checks=True`) catches inconsistencies.

Brief rationale

Lat-lon is the cheapest path to a working global ocean, and the Mercator variant gives isotropic cells without resorting to a distorted grid. MPAS Voronoi is the obvious differentiable analog of MPAS-O ([Ringler et al. 2013]); it gives quasi-uniform global coverage and regional refinement without poles. The pure-C-grid cubed-sphere ocean is the explicit long-term path (memory `project_cgrid_cubesphere_ocean.md`), modelled on MITgcm LLC; the existing C-D grid implementation predates that decision.

Literature pointer

[Ronchi, Iacono & Paolucci 1996], [Putman & Lin 2007], [Ringler et al. 2013] for MPAS; MITgcm LLC for cubed-sphere C-grid precedent.

Known issues / caveats

- **Cubed-sphere ocean** has an exponential face-boundary instability (e-folding ~ 0.8 days on geostrophic adjustment) (memory: `project_cubesphere_ocean_instability.md`). It is excluded from the global wind matrix and from any production scientific result. The root causes are documented as: (a) Lagrange-interpolated binary land mask becomes fractional at panel boundaries, (b) C-D vs A-grid mismatch in PGF stencil at panel boundaries, (c) ocean's $\rho \rightarrow p \rightarrow \mathbf{u} \rightarrow \text{advect} \rightarrow \rho$ loop amplifies it.
 - The spectral ocean exists for completeness, runs into Gibbs at coastlines, and is **not** in the production test matrix.
-

4. Staggering choice

What is configurable in legoESM today

- Lat-lon: **pure C-grid only**. There is a `barotropic_staggering` flag on the *cubed-sphere* `OceanConfig` ("a_grid" or "c_grid") but lat-lon and MPAS are always C.
- MPAS: **pure C-grid** by construction (TRiSK lives on a primal Voronoi / dual Delaunay C-pair).
- Cubed-sphere ocean: **C-D grid** (D-grid winds prognostic; C-grid diagnosed for fluxes), inherited from FV3-style dycore design and scheduled for replacement.

What we currently default to / use in production

C-grid everywhere except the cubed-sphere legacy.

Brief rationale

C-grid is the de-facto production standard for modern ocean models (MOM6, MPAS-O, NEMO, ROMS, MITgcm). The compact one-cell PGF and divergence stencils eliminate the $2\Delta x$ checkerboard null space of A-grid. The decision to *not* use C-D on cubed-sphere (memory `project_cgrid_cubesphere_ocean.md`) follows MITgcm LLC, which has run global ocean for 20+ years at high resolution on pure C-grid cubed-sphere.

Literature pointer

[Arakawa & Lamb 1977], [Sadourny 1975]; MITgcm LLC precedent for cubed-sphere C-grid ocean.

Known issues / caveats

- The **C-grid Coriolis null mode** (Arakawa-Lamb null space: modes with $\nabla \cdot \mathbf{U} = 0$ and $f \cdot \mathbf{V} = 0$ invisible to both continuity and Coriolis) shows up in `ocean_model_latlon_cgrid.py` as the **barotropic mode noise issue** (± 5 cm/s grid-scale alternating-sign noise in V_{baro} that does not average to zero, polluting any momentum-budget diagnostic; memory `project_barotropic_noise_issue.md`; docs/issues/barotropic_mode_noise.md).
- The same mechanism appears on TRiSK as the rotational null branch (Thuburn 2008; Ringler+ 2010 §6); the implicit-CN barotropic solver structurally kills it (memory `project_mpas_barotropic_noise.md`).

5. Discrete operators per grid

What is configurable in legoESM today

Lat-lon C-grid Operators in `ocean/dynamics/latlon_cgrid_operators.py`: - `divergence_cgrid(F_x, F_y, grid)` — finite-volume on cell areas - `gradient_x_cgrid`, `gradient_y_cgrid` — at u/v faces - `curl_vertex_cgrid` — at vorticity points (cell corners) - `laplacian_cgrid`, `bilaplacian_cgrid` — scalar - `vector_laplacian_cgrid`, `vector_bilaplacian_cgrid` — vector - `strain_rate_cgrid`, `smagorinsky_*`, `leith_biharmonic_tendency_cgrid` - `interp_cell_to_uface` (arithmetic), `min_cell_to_uface` / `min_cell_to_vface` (MITgcm hFacW-style **min-rule** for partial cells) - `partial_cell_pgf_correction_x/y` — Adcroft & Campin 2004 - `density_jacobian_pgf_smc03_x/y` — Shchepetkin & McWilliams 2003 - `pv_flux_al81_partial_cell` — Arakawa-Lamb 1981 PV flux variant - Metric corrections: $\cos(\phi)$ factors for sphericity throughout.

MPAS Voronoi (TRiSK) Operators in `core/operators_voronoi.py`: - `divergence_cell` / `divergence_cell_3d` — discrete Gauss on cells - `gradient_edge` / `gradient_edge_3d` — projection along dcEdge-normal - `curl_vertex` / `curl_vertex_3d` — circulation on dual triangles - `tangential_velocity` / `tangential_velocity_3d` — Thuburn 2009 reconstruction - `vertex_thickness_3d` (kite-area mean) / `vertex_thickness_hybrid` (active-renormalized for partial cells, with a tunable threshold) - `potential_vorticity_vertex_3d` -

pv_flux_energy_conserving_3d (Ringler et al. 2010 §3, eq. 38) - pv_flux_enstrophy_conserving_3d (Ringler et al. 2010 §3, eq. 49) - kinetic_energy_cell_3d, vector_laplacian_del2_3d, vector_laplacian_del4_3d - smagorinsky*_3d, leith*_3d - apvm_correction_3d — Sadourny–Basdevant 1985

Partial-cell-aware helpers in ocean/dynamics/mpas_partial_cell_helpers.py: min_cell_to_edge, donor_cell_to_edge, compute_max_level_edge_bot, vertex_thickness_hybrid, partial_cell_pgf_correct, density_jacobian_pgf_smc03_mpas, density_jacobian_pgf_ahh08_mpas.

Cubed-sphere C-D Operators in core/operators_cdgrid.py: D-grid vorticity (exact circulation), Arakawa–Lamb 4-point Bernoulli gradient, center→corner / corner→center interpolation, dgrid→cgrid mass flux, FCT tracer advection (Zalesak 1979).

What we currently default to / use in production

On lat-lon C-grid the production tracer-flux operators use the **min-rule** at faces for partial cells, matching MOM6/MITgcm hFacW. On MPAS, the continuity equation uses **donor-cell upstream** edge thickness for h_e (Petersen 2015 §3.4) while the depth-mean and PV use **min-rule**; on flat-bottom z^* the two agree.

Brief rationale

The partial-cell face-thickness inconsistency (centered $0.5(h_{c1} + h_{c2})$ vs min-rule) is well known to drive a barotropic-baroclinic residual that produces phantom currents at topographic steps; the “Audit 1” finding in project_mpas_etopo_instability.md showed this on the explicit-substep MPAS barotropic path.

Literature pointer

[Arakawa & Lamb 1977], [Thuburn, Ringler, Skamarock & Klemp 2009], [Ringler et al. 2010], [Petersen et al. 2015].

Known issues / caveats

- Cubed-sphere C-D grid: the Arakawa-Lamb 4-point stencil amplifies halo-exchange interpolation error at panel boundaries (memory project_cubesphere_ocean_instability.md); float64 is required for the baroclinic PGF, and even then dynamic runs blow up.
- The vertex_thickness_hybrid=0.5 default on MPAS partial cells was identified in “Audit 3” of project_mpas_etopo_instability.md as a real correctness bug at topographic steps (the min-rule branch amplifies $q = \zeta/h_v$ by $O(h_{\max}/h_{\min})$ at partial-cell step vertices). It was demoted to a knob (vertex_thickness_alpha) defaulting 0.5 for bit-exact backward compat; **production should set 0.0**.

6. Coriolis and PV-flux discretization

What is configurable in legoESM today

MPAS

- `pv_scheme` {"energy", "enstrophy", "mixed"} selects the TRiSK PV flux variant; "mixed" blends with weight `pv_alpha` ([Ringler et al. 2010] §3).
- `apvm_dt > 0` enables the Anticipated Potential Vorticity Method upstream bias on the vertex PV ([Sadourny & Basdevant 1985]; [Ringler et al. 2010] §4.2.3).
- Planetary Coriolis is **not** in the PV flux. It is applied as a separate **forward-backward Matsuno step** on the baroclinic perturbation (`_forward_backward_coriolis_mpas_3d` in `ocean_model_mpas.py:56`), and inside the barotropic loop (`barotropic_mpas.py` / `barotropic_implicit_mpas.py`).

Lat-lon

- `ke_gradient_scheme` {"centered", "hollingsworth"} (default "centered"; "hollingsworth" is the recently added NEMO 4.2.1 `nkeg_HW` form ([Hollingsworth, Kållberg & Renner 1983]; [Arakawa & Hsu 1990]) at `ocean_pe_latlon_cgrid.py:950-982`, commit d0183817).
- Relative vorticity flux $\zeta \times \mathbf{u}$ uses **Arakawa-Lamb 1981 (AL81)** — the 12-point energy-and-enstrophy- conserving triad — with the Le Sommer et al. (2009) / Stewart & Dellar (2016) partial-cell weights (`pv_flux_al81_partial_cell` in `latlon_cgrid_operators.py:2435`, called from `ocean_pe_latlon_cgrid.py:1213`). The simple 2-point Sadourny form was tried earlier and was unstable on ETOPO bathymetry (NaN by day 19).
- Planetary $f \cdot \mathbf{v}$ is split off the PV flux and couples through a **Sadourny 4-point average** in a separate forward-backward Matsuno step (`_forward_backward_coriolis_3d` in `ocean_model_latlon_cgrid.py:274`).

What we currently default to / use in production

- MPAS: `pv_scheme="enstrophy"` (avoids the ζ -checkerboard null mode of the energy-conserving variant). Note: AL81 (the combined energy + enstrophy variant on quadrilateral C-grids) is **not** in the MPAS dispatch today; the three options are "energy", "enstrophy", "mixed".
- MPAS: `apvm_dt = dt` set by the test matrix.
- Lat-lon: AL81 PV flux for ζ is the only option (and the default — `pv_flux_al81_partial_cell` is the wired scheme).
- Lat-lon: `ke_gradient_scheme="centered"` for legacy bit-exactness; **"hollingsworth" is strongly recommended for any stratified realistic-bathymetry run** (issue #263).

Brief rationale

The energy-conserving TRiSK PV flux has a ζ -checkerboard null mode at vertices (Ringler et al. 2010 §4.2.3) that grows on nonlinear interaction; enstrophy-conserving suppresses it but lacks exact KE conservation. APVM is the standard upstream bias for controlling the ζ null mode without sacrificing too much energy. The Hollingsworth correction on the KE gradient is the NEMO-default

fix for the Hollingsworth–Kållberg spurious vortex stretching mode in stratified flow over sloping bathymetry.

Literature pointer

[Sadourny 1975], [Arakawa & Lamb 1981], [Arakawa & Hsu 1990], [Ringler et al. 2010], [Hollingsworth, Kållberg & Renner 1983], [Le Sommer et al. 2009], [Stewart & Dellar 2016] (AL81 partial-cell weights).

Known issues / caveats

- **MPAS Coriolis double-counting (#103)** appeared because the PV flux historically used the *total* velocity, then the barotropic solver applied $f \times \mathbf{u}$ to the depth mean again. Fixed by the perturbation split (memory `project_global_barotropic_wind.md`) at the cost of breaking the Ringler+ energy-conservation identity (issue #160). The 1-level truly barotropic case is the recommended sanity check (test `global_barotropic_wind_1lev`); the 10-level multi-layer reproduction is now believed correct but stays scheduled debt.
- The energy- vs enstrophy-conserving trade-off remains an unresolved scientific question on partial cells: in `project_mpas_etopo_instability.md` §8f the operator-elimination matrix showed `pv_scheme="energy"` and `"enstrophy"` give *bit-identical* day-90 $\max|u|$ on ETOPO+ico-4 — neither variant controls the partial-cell PGF-residual amplification mode.

7. Pressure gradient force

What is configurable in legoESM today

Lat-lon (`LatLonCGridOceanConfig.pgf_scheme`)

Literal	Scheme	Status
"adcroft"	centered $\partial_x p' / \rho_0$ + Adcroft & Campin 2004 face correction	Default
"smc03"	Shchepetkin & McWilliams 2003 density-Jacobian	Available, validated

MPAS (`MPASOceanConfig.pgf_scheme`)

Literal	Scheme	Status
"centered"	bare $\nabla(p' / \rho_0)$	Default (legacy bit-exact)
"adcroft"	+ Adcroft & Campin 2004 face correction (needs CVT)	Production with partial cells + <code>use_h_actual_pgf=True</code>

Literal	Scheme	Status
"smc03"	Shchepetkin & McWilliams 2003 density-Jacobian (column harmonic-slope $\rho(z)$ reconstruction)	Available
"ahh08"	Adcroft, Hallberg & Hill 2008 analytic finite-volume PGF (closed-form $\int p dz$ via Wright EOS rational form)	Available, machine-zero rest-state
"zero"	Drops PGF entirely; diagnostic only	Not for production

The EOS-pressure 2-pass iteration (`iterate_eos_and_pressure_anomaly` in `ocean_tendency_common.py`) computes ρ from $\text{EOS}(T, S, p_{\text{hydro}}(\rho))$ in 2 iterations against the *reference* thickness profile dz_{ref} (i.e. $J = 1$, $\eta = 0$), so that the resulting p' does not double-count the $-g\nabla\eta$ forcing handled by the barotropic solver.

Optional reference profile $\rho_{\text{ref}}(z)$ subtraction: - `use_static_baroclinic_rho_ref=True`: $\rho' = \rho - \rho_{\text{ref}}(z)$ where $\rho_{\text{ref}}(z)$ is frozen at init from `compute_static_rho_ref_z(T_{init}, S_{init})`. - `use_baroclinic_rho_ref=True` (legacy / discouraged): the reference profile is the *dynamic* wet-cell mean recomputed every call. Mutually exclusive; the dynamic variant NaN'd at day 60 on ETOPO+ico-4 (`project_mpas_etopo_instability.md` §8c).

What we currently default to / use in production

- Lat-lon C-grid: `pgf_scheme="adcroft"` with partial cells.
- MPAS: `pgf_scheme="adcroft"` + `use_h_actual_pgf=True` is the production choice — verified bit-equivalent to `smc03` on 5° ETOPO 30-day at 4 sig figs (memory `project_mpas_etopo_instability.md`).

Brief rationale

With z^* and partial bottom cells, all interior cells are horizontally aligned ($\partial_x z_k = 0$); the partial bottom cell at each column is the only place the σ -coordinate-style PGF cancellation problem reappears. Adcroft & Campin's face correction handles this with a per-cell centroid-depth-aligned pressure differencing; SMC03 handles it via a column-aware density-Jacobian reconstruction; AHH08 evaluates $\int p dz$ analytically per cell using the Wright EOS rational form. All three give rest-state $|u_{\text{rest}}| < 10^{-5} \text{ m s}^{-1}$ on ETOPO at ico-4. AHH08 is in fact *exact* at rest by construction; that exactness is the property MOM6 leverages.

Literature pointer

[Adcroft & Campin 2004], [Adcroft, Hill & Marshall 1997], [Shchepetkin & McWilliams 2003], [Adcroft, Hallberg & Harrison 2008].

Known issues / caveats

- **All four PGF schemes hit the same long-run ceiling on ETOPO+ico-4** (memory `project_mpas_etopo_instability.md §8i`): the day-30+ growth is *amplification-dominated*, not seed-dominated. Seed magnitudes range from machine ϵ (AHH08) to 1.4×10^{-6} (Adcroft), but day-90 $\max|u|$ varies non-monotonically with seed — the AHH08 implementation with machine-zero seed is $3.9\times$ worse at day 90 than Adcroft, because Adcroft's 1.4×10^{-6} steady seed creates a viscous-equilibrium attractor and AHH08 removes that attractor.
 - Static $\rho_{\text{ref}}(z)$ is implemented but gives no additional benefit under Adcroft on the production stack — the PGF residual under Adcroft is already below where static $\rho_{\text{ref}}(z)$ would help.
-

8. Equation of state

What is configurable in legoESM today

- `OceanConfig.eos = "wright"` (default) or `"linear"`.
- When `"linear"`: `OceanConfig.eos_linear: LinearEOSConfig` with `rho_ref`, `alpha_T`, `beta_S`, `T_ref`, `S_ref`.

All EOS calls funnel through `make_eos_fn(eos, eos_linear)` in `ocean/eos.py:257`. The Wright form is implemented in `wright_eos(T, S, p)`:

$$\rho(T, S, p) = \frac{p + p_0(T, S)}{\lambda(T, S) + \alpha_0(T, S)(p + p_0(T, S))}$$

with polynomial α_0 , p_0 , λ in T, S ([Wright 1997]). Coefficients are inherited from the MOM6 `MOM_EOS_Wright.F90` implementation. Intermediate arithmetic is promoted to fp64 to avoid catastrophic cancellation in $p_0 \sim 5.8 \times 10^8$ Pa.

Thermal expansion $\alpha_T = -(1/\rho)\partial\rho/\partial T$ and haline contraction $\beta_S = (1/\rho)\partial\rho/\partial S$ are computed by `jax.grad` of the scalar Wright EOS plus `jax.vmap` (`ocean/eos.py:147`).

Ocean-specific constants (centralised in `legoesm.constants` and re-exported by `ocean.eos`): - `rho_0 = constants.rho_ocean = 1025.0` kg/m³ - `c_sw = constants.c_sw - T_freeze_ocean = constants.T_freeze_ocean = 271.35` K (NB: freshwater `T_freeze = 273.15`; this is the only intentional CelsiusKelvin literal exception in the codebase). - `scale_depth = 1000.0` m (reference e-folding depth for stratification IC helpers).

What we currently default to / use in production

Wright 1997 everywhere except small-scale idealised tests (lock exchange, etc.) where linear is appropriate.

Brief rationale

Wright matches MOM6, gives smooth gradients for α_T , β_S , is differentiable and inexpensive. Linear is reserved for tests where analytic answers exist.

Literature pointer

[Wright 1997]; MOM6 implementation (`src/equation_of_state/MOM_EOS_Wright.F90`).

Known issues / caveats

- TEOS-10 / Roquet et al. 2015 polynomial is *not* implemented. For climate-class production work TEOS-10 captures thermobaric and cabbeling effects more accurately. Adding it would be a drop-in `eos_fn` if the polynomial fit were ported.
- Wright is not clipped (per `legoesm.ocean.eos` docstring, issue #165). Out-of-range inputs from advection overshoots or coupler bugs are surfaced as unphysical ρ rather than silently zeroed.

9. Tracer advection

What is configurable in legoESM today

Lat-lon C-grid (`LatLonCGridOceanConfig.tracer_advection`)

Literal	Scheme	File	Notes
"upwind"	1st-order donor cell	<code>ocean_model_latlon_cg</code>	Default fallback to *_points
"tvd"	Van Leer TVD	<code>ocean_model_latlon_cg</code>	Default, two to *points
"ppm"	PPM (Colella-Woodward 1984)	<code>ocean/advection.py</code>	High-order
"ppm_fct"	PPM + Zalesak FCT	<code>ocean/advection.py</code>	Monotone limiter
"dst3"	MITgcm Scheme 33 / DST-3	<code>ocean/advection.py</code>	Direct-space-time
"dst3_multidim"	DST-3 multi-dim	<code>ocean/advection.py</code>	Known unstable for strong flows
"weno5" / "weno7"	Smoothness-optimised WENO (Silvestri 2024)	<code>ocean/advection.py</code>	
"som"	Prather 1986 Second-Order Moments	<code>ocean/advection_som.py</code>	9-moment carry, no limiter, fully differentiable

MPAS (`MPASOceanConfig.tracer_advection`)

Literal	Scheme	Notes
"upwind"	1st-order donor cell (default)	Production
"tvd"	Van Leer TVD with upwind-of-upwind topology cache (<code>compute_upup_cells</code>)	

Vertical advection is flux-form with matched order: TVD vertical when horizontal is TVD; WENO-5/7 vertical when WENO-5/7 horizontal. Vertical w is **diagnosed** from per-layer flux divergence by `diagnose_w_from_flux_div`.

Tracer time integration (`LatLonCGridOceanConfig.tracer_time_integrator`)

Literal	Scheme	CFL note
"euler"	Forward Euler (default)	CFL $\lesssim 1.0$
"ab2"	Adams-Bashforth 2 with MITgcm ABepsBar stabilization	CFL $\lesssim 0.72$
"rk3"	SSP-RK3 in Butcher-tableau form, $3\times$ cost	Conservative; monotonicity not SSP-preserved

Sponge layers and freshwater virtual salt flux

- `ocean/sponge.py:SpongeForcing` carries `gamma`, `T_ref`, `S_ref`, applied via `ocean_tendency_common.apply` with cast to `T.dtype`.
- Freshwater virtual salt flux is in `ocean_tendency_common.apply_freshwater_virtual_salt_top` and is the default freshwater closure (`freshwater_closure="virtual_salt_flux"`); the alternative `"real_freshwater"` is partly wired but the default is virtual.

Implicit-Euler restoring and Q_{sr} split (commit 50d65485)

The surface restoring module (`ocean/physics/surface_forcing/restoring.py`) now supports: - Arbitrary user-supplied 2-D targets `T_star_array`, `S_star_array` (overrides the cosine / constant built-ins); - Q_{sr} subtraction (`subtract_qsr=True`): subtract `sw_down` from the restoring tendency in the surface layer (NEMO/DINO eq 8 non-solar split); - `implicit=True`: analytical implicit-Euler integration of the restoring update, stable for any dt — necessary for strong restoring combined with strong vertical mixing (the DINO-like parameter regime).

What we currently default to / use in production

TVD on lat-lon; TVD on MPAS. The Eady experiment at coarse resolution (10–40 km cells, $L_d \sim 30$ km barely resolved) showed all less-diffusive schemes (DST-3, PPM, PPM-FCT) trigger exponential velocity growth because TVD's implicit diffusion ($\sim 2000 \text{ m}^2/\text{s}$ at fronts) was acting as an essential

sub-grid eddy parameterization (memory `project_dst3_advection.md`). The fix is **GM/Redi** at coarse resolution, not less-diffusive advection.

Brief rationale

TVD is the cheapest fully-monotone, fully-differentiable scheme. SOM (Prather 1986) achieves $\sim 10^3 \times$ less spurious diapycnal mixing than TVD via 9-moment polynomial sub-cell distributions; the absence of a limiter makes it cleanly differentiable ([Hill et al. 2012]; memory `project_som_advection.md`).

Literature pointer

[Colella & Woodward 1984], [Zalesak 1979], [Prather 1986], [Hundsdoerfer et al. 1995] (DST-3), [Silvestri et al. 2024] (WENO-ILES).

Known issues / caveats

- AB2 + FCT/PPM_FCT: the individual time-level flux divergences F^n, F^{n-1} are monotone, but their AB2 linear combination is **not** guaranteed monotone — same limitation as MITgcm (documented in `_compute_advection_flux_div` docstring).
- SSP-RK3 in Butcher form conserves mass exactly but does **not** preserve the Shu-Osher SSP property under nonlinear limiters (TVD/WENO/FCT); new extrema may appear.
- `dst3_multidim` is unstable for strong flows — kept available but not in the production matrix.

10. Momentum advection

What is configurable in legoESM today

Lat-lon C-grid (`LatLonCGridOceanConfig.momentum_advection`)

Literal	Scheme
"vector_invariant"	KE-gradient + relative PV flux + vertical advection (default)
"weno5"	Silvestri et al. 2024 WENO-5 momentum, with optional D-term
"weno7"	Silvestri et al. 2024 WENO-7 momentum

`weno_d_term: bool = True` enables the Silvestri Eq. (31)–(32) divergence flux (D term) for ILES dissipation; matching-direction divergence is WENO-upwinded, cross-direction stays centered (Silvestri Appendix C).

`ke_gradient_scheme` is independent: "centered" or "hollingsworth", as documented in §6.

MPAS Vector-invariant only; the momentum advection is via the TRiSK PV flux + KE-gradient. WENO momentum advection is not implemented on MPAS Voronoi (the smoothness-optimised stencils require a regular structured neighbourhood that's awkward on Voronoi).

What we currently default to / use in production

- Lat-lon: vector-invariant; switch to Hollingsworth KE-gradient on realistic stratified bathymetry (issue #263 fix; commit d0183817).
- MPAS: vector-invariant.

Brief rationale

Vector-invariant matches MOM6 and MPAS-O. WENO-ILES (Silvestri 2024) is implemented for the rotational mesoscale-permitting regime where explicit Smagorinsky-style closure damps too aggressively. The Hollingsworth KE-gradient correction is the NEMO 4.2.1 fix for spurious vortex stretching over sloping bathymetry — important for the global overturning experiment.

Literature pointer

[Sadourny 1975], [Hollingsworth, Kållberg & Renner 1983], [Silvestri et al. 2024], [Arakawa & Hsu 1990].

Known issues / caveats

- WENO momentum advection removes 1st-order implicit viscosity and may need KPP or Richardson-dependent A_v for stability — known trade-off (memory `project_weno_iles.md`).
- WENO + KE-gradient (K term in Silvestri Eq. 33) replaces $(\bar{u})^2$ with $\bar{u}^2$ via WENO5 reconstruction; adds $O((\Delta U)^2)$ shock-capturing KE dissipation.

11. Time integration of the slow (3-D) modes

What is configurable in legoESM today

Per-step pipeline (lat-lon C-grid) `LatLonCGridOceanModel._step_impl` in `ocean_model_latlon_cgrid.py` runs: 1. **Compute baroclinic tendencies** (everything except planetary Coriolis). 2. **Update tracers** with forward Euler (modulated by AB2/RK3 for advection sub-step). 3. **Split slow forcing**: $F_{\text{slow}} = \text{depth-mean of } du/dt_{\text{baroclinic}}$, applied online inside the barotropic substep; $du/dt_{\text{pert}} = du/dt - F_{\text{slow}}$ applied to 3-D before barotropic. 4. **Add depth-mean biharmonic damping** A_2 on $\bar{U} / \bar{V}$ if `B_h_barotropic > 0`. 5. **Apply forward-backward Matsuno Coriolis** on the baroclinic perturbation. 6. **Run barotropic substeps** (explicit) or **Crank-Nicolson PCG Helmholtz solve** (`implicit_cn`). 7. **Reconcile 3-D velocity** with the time-averaged $H\bar{u}_{\text{avg}}$ from the barotropic solver. 8. **Flux-form tracer advection** using the corrected 3-D velocity; horizontal flux + vertical flux with w diagnosed from per-layer flux divergence.

9. **GM/Redi** tracer tendency (after the diabatic step). 10. **Freshwater virtual salt flux** at top layer. 11. **Implicit (backward-Euler) vertical mixing** if `implicit_vertical_mixing=True`. 12. **Conservation fixers** if `use_conservation_fixer=True` (default `False`).

Outer time integrators (generic) `legoesm/timestepping/dispatch.py:_INTEGRATORS`: - `"ssp_rk3"`, `"ssp_rk34"`, `"ssp_rk54"`, `"rk4"`. Used by the *spectral* ocean (SSP-RK3 default) and the cubed-sphere atmosphere/SW dycores. The lat-lon and MPAS ocean models use the in-place `step()` pipeline above with forward Euler at the outermost layer; the staged pipeline integrates each sub-process with its own appropriate method.

Segment-loop architecture for AD

The training-mode and validation harness build trajectories via `build_segment_fn(...).raw` (non-JIT, non-donating) inside `eqx.filter_value_and_grad`. This avoids buffer donation conflicts with reverse-mode AD. `SegmentCarry` carries hot-loop state (e.g. CFL diagnostics zeroed at segment start, not accumulated). MPI halo exchanges use `_sendrecv_vjp` in `parallel/halo_exchange.py:114-133`, a `@jax.custom_vjp` that swaps source/destination in the backward pass — enables `jax.grad` through MPI halo. See `CLAUDE.md` for the discipline.

What we currently default to / use in production

The “Outer Euler + forward-backward Coriolis + split-explicit or implicit-CN barotropic + flux-form tracer at the end” pipeline as above. AB2 tracer advection is used in some integrations; RK3 tracer advection is reserved for high-CFL transient experiments.

Brief rationale

Operator-split per-process integration matches MOM6/MPAS-O conventions and gives a natural place for diabatic processes (implicit-Euler vertical mixing, restoring, GM/Redi) downstream of the dynamic substep.

Literature pointer

[Adcroft et al. 2019] (MOM6 `MOM_dynamics_split_RK2`), [Higdon 2005], [Hallberg 1997].

Known issues / caveats

- We do **not** have a generic Adams-Bashforth 3 outer integrator.
- We do **not** have a fully implicit-vertical *and* implicit- barotropic stepper of the MITgcm-style; the implicit components are layered onto the outer Euler split.
- The outer forward-Euler step plus forward-backward Coriolis is not 2nd-order accurate (it’s $O(dt)$). This is acceptable for the resolutions and timescales we target but is a known limitation to flag to Alistair.

12. Barotropic–baroclinic mode splitting

What is configurable in legoESM today

Solver selection

- `barotropic_solver` {"explicit_substep", "implicit_cn"} (lat-lon C-grid and MPAS).

Explicit substep

- `n_barotropic_substeps` (default 30).
- `bebt`: float [0,1] — backward-Euler / backward-time blend for the PGF [Shchepetkin & McWilliams 2005]. Default 0.2 (MOM6 convention). 0 = pure forward-backward.
- `barotropic_time_filter` {"cosine", "box"} — time filter applied to barotropic accumulators. Default "cosine" (Hanning bell, $w_i = 1 + \cos(2\pi(i - n/2)/n)$). Helper at `barotropic_common.compute_filter_weights`.
- `maxvel_barotropic`: float = 0.0 — symmetric clip of $\bar{U}, \bar{V}$ to suppress runaway single-point velocities; MOM6 uses 6 m/s.
- `barotropic_diffusion_alpha`: float — 2-D Laplacian damping on barotropic substeps, dt_s/dt_{ref} -scaled.
- `barotropic_div_damp`: float — divergence damping on the barotropic velocity field (operator dormant by default; recommended 0.05 as a band-aid; memory project_barotropic_noise_issue.md).
- `bottom_drag_r` > 0 triggers an implicit per-substep multiplier $1/(1 + dt_s \cdot r / \max(H, \epsilon))$ via `implicit_bottom_drag_factor` in `ocean_tendency_common.py`.
- `differentiable_barotropic`: True uses `lax.scan`, False uses `fori_loop` (faster, not AD-safe).
- `F_slow_u`, `F_slow_v`, `F_slow_eta` carry the depth-mean baroclinic tendency and the freshwater mass flux into the substep loop — applied each substep so the slow forcing couples to the evolving barotropic state (MOM6 pattern).

Implicit Crank–Nicolson

- `barotropic_implicit_theta_eta`, `barotropic_implicit_theta_pgf`: both default 0.55 (slightly past CN — implicit damping of checkerboard while staying close to 2nd-order). Standard MITgcm/MPAS-O choice.
- `barotropic_implicit_pcg_tol`: float = 1e-10, `barotropic_implicit_pcg_maxiter`: int = 200.
- The Helmholtz operator $[I - \theta_\eta \theta_p dt^2 g \nabla \cdot (H \nabla)] \eta^{n+1}$ is solved via `jax.scipy.sparse.linalg.cg` (differentiable via the implicit-function-theorem custom-VJP).
- On MPAS, biharmonic / Laplacian damping on $\bar{U}$ is wired through `barotropic_u_viscosity` and `barotropic_u_biharmonic` (default 0; recommended $\sim 3 \times 10^6$ m²/s and $\sim 10^{15}$ m/s respectively for ico-4 ETOPO).

Tracer flux reconciliation (Hallberg & Adcroft 2009) The barotropic solver returns the time-averaged depth-integrated transport $\bar{H}u_{\text{avg}}$. The tracer flux is built from a *barotropically corrected* 3-D velocity $u_{\text{corrected}} = u_{3D} + (\bar{H}u_{\text{avg}} - Hu_{3D})/H_u$ so that

$\sum_k h_k u_{\text{corrected},k} \equiv \bar{H} u_{\text{avg}}$. This preserves baroclinic shear while matching the barotropic flux exactly. See `ocean_model_latlon_cgrid.py:864-880` and `ocean_model_mpas.py:618-632`.

What we currently default to / use in production

- **MPAS:** `barotropic_solver="implicit_cn"` is the production default for global overturning and any multi-year run (memory `project_mpas_barotropic_noise.md`: validated 5-yr at ico-4 with σ_{grid} plateau at 1.93×10^{-2} m/s, vs explicit's growth to 9.93×10^{-2}).
- **Lat-lon:** `barotropic_solver="explicit_substep"` is still default for backward compat; the implicit-CN path exists and is recommended for noise-sensitive diagnostics.

Brief rationale

- The Hallberg 1997 / Higdon 2005 result that the barotropic must be **time-filtered** before coupling back to the baroclinic step is the reason for the cosine filter.
- The Shchepetkin & McWilliams 2005 weighted time-averaging is the ROMS-style filter our cosine implements at first order.
- BEBT=0.2 is the MOM6 semi-implicit PGF blend; commit 3d0170d was the original Eady stability fix (memory `project_barotropic_solver.md`).
- Implicit-CN was added because the explicit substep with cosine filter has the C-grid Coriolis null-mode pathology that none of Stage 0/1/2 fixes (div-damp, doubled boxcar, ROMS power-law) resolved on lat-lon's Drake-band V_{baro} residual (`docs/issues/barotropic_mode_noise.md`).

Literature pointer

[Hallberg 1997], [Higdon 2005], [Shchepetkin & McWilliams 2005], [Killworth, Webb, Stainforth & Paterson 1991], [Hallberg & Adcroft 2009].

Known issues / caveats

- **Lat-lon barotropic mode noise is OPEN** (memory `project_barotropic_noise_issue.md`): ± 5 cm/s grid-scale noise in time-mean $\bar{V}$ that doesn't average to zero in 50,000 timesteps; amplifies via $\rho H f$ to ~ 0.10 Pa westward force on momentum-budget diagnostics. **Implicit-CN is the only validated fix**; we should likely promote it to default on lat-lon too.
- **Drag double-counting in explicit-substep** (`implicit_bottom_drag_factor` docstring): the explicit barotropic loops apply the implicit drag multiplier *in addition to* the depth-mean drag carried in F_{slow} (which is the depth-average of the 3-D solver's du/dt that already includes bottom-cell drag). Effective barotropic-mode drag is $\approx 2r/H$ rather than r/H . Implicit-CN does NOT have this issue (it relies on F_{slow} alone).
- Cosine time filter degenerates to zero at `n_substeps == 1`; the helper falls back to box (codex iter-1 #4).

13. Horizontal dissipation

What is configurable in legoESM today

Knob	Operator	Lat-lon	MPAS
A_h	$A_h \nabla^2 \mathbf{u}'$		
A_h_lat_scaling, A_h_floor	$\cos(\phi)$ scaling		(n/a)
A_h_eq_boost, A_h_eq_sigma_deg	Gaussian boost at equator		(via equatorial_visc_boost)
B_h	$B_h \nabla^4 \mathbf{u}'$		
B_h_barotropic	$-\nu_4 \nabla^4 \bar{U}$		(via barotropic_u_biharmonic)
C_smag	Biharmonic Smagorinsky		
C_smag_lap	Laplacian Smagorinsky		
C_leith, C_leith_modified	Leith biharmonic		
K_zeta_bih	$-K_\zeta \nabla^4 \zeta$	(n/a)	
K_h, K_bih	Tracer Laplacian / biharmonic		
barotropic_diffusion_alpha	Laplacian damping on barotropic substep		
barotropic_div_damp	Div-damping on $\bar{U}$		
barotropic_u_viscosity	$A_{\text{baro}} \nabla^2 \bar{U} / -\nu_4 \nabla^4 \bar{U}$	(via B_h_barotropic)	
barotropic_u_biharmonic			
equatorial_visc_boost	$A_{\text{eff}} =$	(via A_h_eq_boost)	
equatorial_visc_sigma_deg	$A_{\text{eff}} + b \exp(-(\phi/\sigma)^2)$		
slope_foot_alpha, slope_foot_threshold, slope_foot_n_levels	Locally enhanced A_h over steep slopes (MOM6 OM4 KH_BG_2D analog)		—
hyperdiff_coeff, hyperdiff_order	High-order spectral hyperdiff	(spectral only)	(n/a)

The biharmonic operator on $\mathbf{u}'$ uses the `vector_laplacian_del4_3d` = $-\nabla^2(\nabla^2 \mathbf{u})$ convention.

What we currently default to / use in production

Production ETOPO+ico-4 stack (memory project_mpas_etopo_instability.md): - A_h $1 \times 10^6 \text{ m}^2/\text{s}$; B_h 10^{15} m/s - equatorial_visc_boost = 5, equatorial_visc_sigma_deg = 5 - barotropic_u_viscosity = $3 \times 10^6 \text{ m}^2/\text{s}$ - apvm_dt = dt, K_zeta_bih 10^{14} (if used) - bottom_drag_r 2.5×10^{-3} , bottom_drag_bbl_thickness = 50 m - A_v

1×10^{-2} (vertical, with implicit solver)

Brief rationale

The reason the matrix is this baroque is that the bottom-trapped mode at MPAS+ETOPO is the model's load-bearing scientific limitation today (§13 caveats). Each lever was tested in the §8 damping sweep of `project_mpas_etopo_instability.md`: - Pure B_h biharmonic up to 10^{14} : **0% reduction** — structurally null at large scales for this mode; - Smagorinsky $C=0.1, 0.15$: **0%** (same as B_h); - $A_h \times 3 = 3 \times 10^6$: $2.55\times$; - Combined ($A_h + A_v + \text{drag}$): **$5.24\times$** .

Literature pointer

[Smagorinsky 1963], [Griffies & Hallberg 2000], [Leith 1996], [Fox-Kemper & Menemenlis 2008], [Killworth & Edwards 1999] (BBL drag), [Rhines 1969] (bottom-trapped wave background).

Known issues / caveats

MPAS+ETOPO bottom-trapped instability is an algorithm-class limit in the current dycore. Damping toolkit and higher resolution (ico-5) have been exhausted (memory `project_mpas_etopo_instability.md` §8g–§8k). The production-feasible spinup window at ico-4 is **~90 days** under the combined-damping config. For multi-year production, an algorithmic change is required (candidates: terrain-following coordinate; analytic FV PGF with attractor-preserving design; or simply accept ico-5+).

14. Vertical mixing

What is configurable in legoESM today

`VerticalMixingConfig.scheme` {"none", "constant", "richardson", "kpp"}.

Schemes

- "constant": A_v, K_v specified explicitly (`ConstantVerticalMixingConfig`).
- "richardson": Pacanowski & Philander 1981 with $K_0, \alpha, n, K_{bg}, A_{bg}, Pr_t$.
- "kpp": LMD94 KPP boundary-layer (`physics/vertical_mixing/kpp.py`). Parameters in `KPPConfig`:
 - $Ri_{crit} = 0.3$ (matches NCAR POP2 / MOM6 default);
 - $C_v = 1.6$ (unresolved-shear coefficient);
 - $\gamma_T = \gamma_S = 6.33$ (non-local transport coefficients);
 - $K_{conv} = 1.0, Ri_{conv} = 0.0$ (convective limit);
 - $K_0_{shear} = 5e-3, Ri_0 = 0.7$ (LMD94 interior shear mixing);
 - $c_s = 98.96, c_b = 0.599, \epsilon_{lmd} = 0.1$ (LMD94 constants);

- `crossing_sharpness = 20.0`, `crossing_threshold = 0.1` (sigmoid for h_{bl} blend — fully differentiable, no iterative root-finding).

Convective adjustment (`OceanConvectionConfig`)

- `"enhanced_diffusion"` — sets $K_v = K_{\text{conv}} = 1 \text{ m}^2/\text{s}$ (smoothed by sigmoid in $\partial_z \rho$);
- `"plume"` — entraining mass-flux plume (Prather-like vertical scan; `physics/convection/plume.py`).

Implicit vertical solver `implicit_vertical_mixing: bool` (defaults: `True` on MPAS, `False` on lat-lon) routes A_v on $\mathbf{u}$ and K_v on T, S through a backward-Euler tridiagonal solve (`physics/vertical_mixing/implicit_solver.py`). Removes the explicit-diffusion CFL limit $dt < dz^2/(2K)$ which becomes binding when $K_{\text{conv}} = 1 \text{ m}^2/\text{s}$ is active with surface $dz < 30 \text{ m}$ — matches MOM6/NEMO/POP/MITgcm convention.

What we currently default to / use in production

KPP + enhanced-diffusion convection + implicit vertical mixing.

Brief rationale

KPP is the production standard ([Large, McWilliams & Doney 1994]). Implicit vertical solve is needed to absorb K_{conv} without a tiny dt . The sigmoid-based h_{bl} diagnosis avoids iterative root-finding — needed for `jax.grad` smoothness.

Literature pointer

[Large, McWilliams & Doney 1994], [Van Roekel et al. 2018] (CVMix), [Pacanowski & Philander 1981] (Richardson).

Known issues / caveats

- Langmuir enhancement (Li et al. 2016, Van Roekel et al. 2012) is **not** wired.
- TKE (Gaspar et al. 1990) / ePBL (Reichl & Hallberg 2018) / Mellor-Yamada 2.5 are **not** implemented.
- Tidal mixing (Simmons et al. 2004; St. Laurent et al. 2002) is **not** wired.
- The convective plume is in `physics/convection/plume.py` but the production stack uses `enhanced_diffusion` — the plume has not been validated against a deep-convection benchmark.

15. Mesoscale eddy parameterization (GM/Redi)

What is configurable in legoESM today

`LateralMixingConfig.scheme` {"none", "harmonic", "biharmonic", "gm_redi"}.

`GMRediConfig`: - `kappa_GM`, `kappa_Redi` — thickness and isopycnal diffusivities (defaults 10^3 m²/s each). - `S_max` — slope clipping for DM95 tapering (default 0.01). - `visbeck`: `VisbeckConfig` — adaptive coefficient $\kappa_{\text{Visbeck}} = \alpha L^2 \langle N|S \rangle_z$ ([Visbeck et al. 1997]); `alpha` = 0.015, `L_fixed` = 100 km or first-baroclinic Rossby radius ($NH/|f|$) with `|f|`-floor; clipped to [`kappa_min`, `kappa_max`]. - `slope_scheme` {"centered", "triads"} (lat-lon C-grid only).

Lat-lon C-grid GM/Redi Both "centered" and "triads" slope discretisations are implemented (`gm_redi_latlon_cgrid.py`). The triad scheme is [Griffies, Gnanadesikan, Pacanowski et al. 1998] with 4 quarter-cell triads per u-face / v-face and 8 triads per w-face, each using the SAME three $\rho/T/S$ values for both slope and gradient so the Redi flux vanishes exactly for tracers constant along isopycnals (memory `project_gm_redi_latlon.md`).

MPAS GM/Redi Centered only (`gm_redi_mpas.py`); the triads branch raises `NotImplementedError` — Phase 5 of `docs/ocean_experiments/gm_redi_mpas_plan.md`.

What we currently default to / use in production

- Lat-lon: `slope_scheme="triads"`, `kappa_GM` = `kappa_Redi` = 1000 m²/s. The triad implementation residual is $\sim 4 \times 10^{-16}$ K/s at machine precision (memory `project_gm_redi_latlon.md`).
- MPAS: `slope_scheme="centered"`, same κ .

Brief rationale

- GM 1990 + Redi 1982 are the production standard for eddy-parameterising coarse runs ([Gent & McWilliams 1990], [Redi 1982], [Griffies 1998]).
- Triads are required for century-scale runs because centered Redi has a small but cumulative cross-isopycnal residual that triads cancel algebraically per stencil. We saw a 10^7 x improvement in Redi-only Eady tests (memory `project_gm_redi_latlon.md`).
- Visbeck adaptive κ couples diffusivity to local growth rate, important for representing eddy variability in highly baroclinic regions (memory `project_gm_redi_latlon.md`).

Literature pointer

[Gent & McWilliams 1990], [Redi 1982], [Griffies 1998], [Visbeck et al. 1997], [Danabasoglu & Marshall 2007], [Fox-Kemper, Ferrari & Hallberg 2008].

Known issues / caveats

- **GM/Redi triads on MPAS Voronoi do not exist anywhere correctly to our knowledge.** Question to Alistair.
 - DM95 taper is applied **on the per-triad whole flux** (diagonal – off-diagonal together), not on the slope. Tapering the slope before combining leaves a residual $K_R(1 - \text{taper}) \cdot \nabla q$ that accumulates dynamically (memory `project_gm_redi_latlon.md`).
 - Density and tracer must be **Neumann-filled inside the triad function**. Land columns with sentinel values destroy the cancellation property (memory `project_gm_redi_latlon.md`).
 - Fox-Kemper submesoscale restratification (Fox-Kemper, Ferrari & Hallberg 2008) is **not** wired.
 - MEKE (Jansen et al. 2015) / GEOMETRIC (Marshall, Maddison & Berloff 2012) prognostic eddy energy are **not** wired.
-

16. Bottom drag

What is configurable in legoESM today

- `BottomDragConfig.scheme` {"none", "linear", "quadratic"}.
- Linear: $\tau_b = \rho_0 r \mathbf{u}_b$, with r [m/s] so that bottom stress is independent of vertical resolution (MITgcm `bottomDragLinear` convention). Default $r = 1.1\text{e-}3$ m/s.
- Quadratic: $\tau_b = -C_d |\mathbf{u}_b| \mathbf{u}_b$, with $C_d = 2.5\text{e-}3$.
- Quadratic-with-floor (`bottom_drag_bg_velocity > 0`): MOM6 `DRAG_BG_VEL` form, $r_{\text{eff}} = (r/u_{bg}) \sqrt{u^2 + u_{bg}^2}$, recovers linear at $|u| \rightarrow 0$ and quadratic at high speed.
- Distributed BBL (`bottom_drag_bbl_thickness > 0`): Killworth-Edwards 1999 / MOM6 `BBL_thick_min` form. Spreads the drag stress over a fixed Ekman thickness H_{BBL} near the seafloor so that thin partial cells (down to ~0.3 m on ETOPO+ico-4) are not pushed beyond CFL. The per-face-column algorithm is in `ocean_tendency_common.bbl_distributed_drag_face_column` and is grid-agnostic.
- Implicit Eulerian drag inside the barotropic substep loop: $u_{\text{new}}/u_{\text{old}} = 1/(1 + dt_s r / \max(H, \epsilon))$ via `implicit_bottom_drag_factor`. Unconditionally stable.

What we currently default to / use in production

- Linear, $r \approx 1.1\text{--}2.5 \times 10^{-3}$ m/s.
- BBL thickness $H_{\text{BBL}} = 50$ m for realistic bathymetry runs.
- Implicit barotropic substep multiplier always-on when `bottom_drag_r > 0`.

Brief rationale

Linear-with-BBL avoids the CFL hazard at thin partial cells on ETOPO+ico-4 (the thinnest bottom cell is 0.28 m, so $r \cdot dt$ at $r = 1.1 \times 10^{-3}$, $dt = 500$ s gives $r \cdot dt = 0.55$ m $> 2 \times$ the cell thickness; explicit drag *sign-reverses* u). BBL formulation bounds per-cell drag tendency by ru/H_{BBL} .

Literature pointer

[Killworth & Edwards 1999], MOM6 BBL_thick_min ([Adcroft et al. 2019]).

Known issues / caveats

- **Double-counting in explicit barotropic path** (§12 caveats): the implicit substep multiplier is applied *in addition to* the depth-mean drag in F_{slow} . Implicit-CN avoids this by construction. This is open architectural debt (implicit_bottom_drag_factor docstring).
- MOM6's implicit barotropic drag (memory project_mom6_slow_forcing.md): MOM6 wires drag into the barotropic Helmholtz solver as an implicit term. Our implicit-CN solver does not currently include drag in the Helmholtz operator (it relies on F_{slow}). Question for Alistair: is this stable enough or do we need to follow MOM6's design?

17. Surface forcing slots (ocean side only)

The ocean exposes the following coupler-side forcing pytree (ocean.state.OceanSurfaceForcing):

```
OceanSurfaceForcing(  
    sw_down,      # downwelling SW [W/m²]  
    q_net,        # net surface heat flux [W/m²]  
    tau_x,        # zonal wind stress [Pa]  
    tau_y,        # meridional wind stress [Pa]  
    freshwater,   # net freshwater flux into ocean [kg/m²/s]  
)
```

The freshwater is broken out into precip / evap / runoff / ice fields by FreshwaterForcing in ocean/freshwater.py.

Forcing schemes (SurfaceForcingConfig.scheme)

Literal	Behaviour
"none"	No surface tendency
"prescribed"	Fixed τ , Q_{net} , $E - P$; wind profile choice in PrescribedForcingConfig.wind_profile {"constant", "cosine_latitude", "single_gyre", "double_gyre", "double_gyre_sin2", "double_gyre_tapered", "global_wind"}
"restoring"	T/S restoring to target profile; new in commit 50d65485 (issue #266): arbitrary 2-D T_star_array/S_star_array, optional subtract_qsr (NEMO/DINO eq. 8 non-solar split), optional implicit=True analytical implicit-Euler

Literal	Behaviour
"combined"	Prescribed wind + T/S restoring (for realistic baroclinic gyre experiments)
"bulk_formulas"	COARE-like (<code>bulk_scheme</code> {"constant", "coare3", "large_yeager"})

Shortwave penetration

`shortwave_penetration_tendency` in `ocean/physics/shortwave_penetration.py` distributes `sw_down` through the water column with a Paulson & Simpson 1977 two-band exponential, configurable by Jerlov water type (`water_type` {"I", "IA", "IB", "II", "III"}; default "II"). Default global average.

Sponge layers

`ocean/sponge.py:SpongeForcing(gamma, T_ref, S_ref, u_ref, v_ref)`, applied through `ocean_tendency_common.apply_sponge_tracer_relaxation` and the similar momentum sponge in the lat-lon PE step. Coefficient helpers (`compute_sponge_gamma_latlon`, `compute_sponge_gamma_mpas`) provide quadratic ramps.

What we currently default to / use in production

- AMIP coupling: bulk formulas (Large-Yeager) for ocean→atm fluxes, with prescribed atmospheric state.
- Idealised channel runs (Eady, ACC): prescribed wind + sponge layers at meridional walls.
- Global overturning: bulk formulas + T/S restoring with arbitrary 2-D targets (commit 50d65485).

Brief rationale

The restoring extension (issue #266) was prompted by the DINO-class setup which needs paper-equation T/S targets (e.g., equatorial Gaussian dip), Q_{st} -split flux convention, and implicit-Euler restoring for any- dt stability under strong restoring + strong mixing. All three were not expressible in the previous cosine- profile-only restoring.

Literature pointer

[Paulson & Simpson 1977], [Large & Yeager 2009], NEMO DINO setup.

Known issues / caveats

- Real freshwater (`freshwater_closure="real_freshwater"`) is partially wired but virtual salt flux is the production default.

- Land-runoff (river input) and ice-melt flux are part of the `FreshwaterForcing` container but the realistic data pipelines for them are coupler-side and not in the ocean codebase.

18. Bathymetry and land treatment

What is configurable in legoESM today

`BathymetryConfig` in `ocean/bathymetry.py`: - `source`: `str` — "idealized", NetCDF/Zarr file, or "etopo"; - `smoothing_passes`: `int` — Laplacian smoothing of H_{bathy} ; - `r_factor_max`: `float` — Mellor-Ezer-Oey r-factor cap (limits $|\Delta H|/H$ between adjacent cells; used in `apply_meo_r_factor_cap` / `apply_meo_r_factor_cap_voronoi`); - `min_water_column_m` — minimum total water column to keep a column wet; - `enforce_straits`: `list` — list of (lat, lon) tuples that must be wet through a specified depth (`CRITICAL_STRAITS` provides Gibraltar, Bering, Florida, Mozambique, etc.); - `fill_isolated_basins`: `bool` — flood-fill to keep only the largest connected ocean.

Land treatment: - `land_mask` at cell centers + `u_mask`, `v_mask` at faces, all separate fields of the state (§3). - **Atomicity invariant**: `compute_face_masks(land_mask)` must hold; runtime check available (`_assert_runtime_invariants` in `ocean_model_latlon_cgrid.py:1283`). - `replace_land_mask(state, new_mask)` in `ocean/init_latlon_cgrid.py:306` atomically rebuilds all three.

For partial cells: the per-level mask `is_active` from `OceanPartialCellCoordinate.is_active` is the canonical below-seafloor gate. Tracer updates, GM/Redi gating, vertical mixing solvers all use `active_3d` rather than a 2-D `mask_3d-broadcast` — the lat-lon path landed this in PR #231, and the MPAS analog (`memory_project_mpas_etopo_instability.md` “Audit 2”) gave a $3.6\times$ day-30 reduction in $\max|u|$ on ETOPO+ico-4.

What we currently default to / use in production

- ETOPO 1° or 0.2° preprocessed by `init_ocean_bathymetry`, with MEO `r_factor_max` 0.15–0.2, smoothing passes 2–20 depending on grid, partial cells on.
- Critical-straits enforcement on global runs.

Brief rationale

Partial cells + MEO r-factor cap is the production combination at 2.5° to 1° global; it’s the same combination MOM6 / NEMO-zps / MITgcm use ([Adcroft, Hill & Marshall 1997]; [Pacanowski & Gnanadesikan 1998]).

Literature pointer

[Adcroft, Hill & Marshall 1997], [Pacanowski & Gnanadesikan 1998], [Mellor, Oey & Ezer 1998].

Known issues / caveats

- The active-mask atomicity invariant is not enforced at the type level — it’s a runtime check + a `replace_land_mask` helper. Bug-class “stale face mask” recurs and is listed in `CLAUDE.md`.
 - MEO with $r_{\text{factor}} = 0.15$ already eliminates fine continental-shelf structure; for $\frac{1}{4}^\circ$ resolution the cap is binding in a scientifically meaningful way.
-

19. Halo exchange and parallelism

What is configurable in legoESM today

- **Native 4-D halo:** `pad_halo_4d` / `pad_halo_vector_4d` (`grids/halo.py:711, 1029`) exchange all vertical levels in one MPI message. All 3-D operators go through the 4-D path; no `vmap(pad_halo)` over levels.
- **MPI VJP wrapper:** `_sendrecv_vjp` in `parallel/halo_exchange.py:114-133` is a `@jax.custom_vjp` that swaps source/destination in the backward pass. Enables `jax.grad` through MPI halo exchange.
- **Scatter–local–gather pattern:** `initialize_distributed(global_n=N) → scatter_to_local()` → rank-local stepping → `gather_to_global()` for I/O only. Both `ModelDriver` and the benchmark scripts use this path. **Never** create full global state per rank.
- **Reductions:** `global_sum_mpi` (allreduce SUM) full VJP support. `global_max_mpi`, `global_min_mpi`, `allgather`, `bcast` are NOT differentiable — diagnostic-only.

What we currently default to / use in production

MPI distributed runs on a small device cluster (Levante and Apple Silicon scaling benchmark `scripts/run_levante_gpu_scaling.py`).

Brief rationale

Halo VJP is the load-bearing AD primitive for distributed gradient training. Without it, every *jax.grad* across MPI would silently miss boundary-condition sensitivities.

Literature pointer

n/a (legoESM-original wrapper).

Known issues / caveats

- `global_max_mpi`, `global_min_mpi` look usable but are **not** differentiable. Keep them out of any loss function. `CLAUDE.md` enforces this explicitly.
- Device mesh creation under MPI: pass per-rank device count to `create_device_mesh()`, not total. The function warns when clamping.

20. Differentiability and JAX considerations

What is configurable in legoESM today

- All ocean state containers are `NamedTuples` registered as JAX pytrees (`ocean/state.py`).
- All operators are pure functions; no mutation.
- All loops are `jax.lax.scan` (AD-safe) or `jax.lax.fori_loop` (faster, not AD-safe). The barotropic loop has a knob (`differentiable_barotropic`) to switch between them.
- The implicit-CN barotropic solver uses `jax.scipy.sparse.linalg.cg` (differentiable via the implicit-function theorem custom-VJP).
- KPP boundary-layer depth uses a smooth sigmoid blend on the bulk Richardson crossing (`KPPConfig.crossing_sharpness = 20.0`, `crossing_threshold = 0.1`) rather than iterative root-finding — fully differentiable.
- GM/Redi slope tapering: DM95 taper applied on the per-triad *whole* flux, not on the slope before combining. Smooth tanh.
- EOS partial derivatives (α_T , β_S) are computed by `jax.grad` of the scalar Wright EOS + `jax.vmap`.
- SOM advection has no limiter and is fully differentiable (memory `project_som_advection.md`).
- The model-wide segment-loop architecture uses `build_segment_fn(...).raw` (non-JIT, non-donating) inside `eqx.filter_value_and_grad` to avoid buffer-donation conflicts with reverse-mode AD.

What is novel

- **End-to-end `jax.grad`** through MPI halo exchange, implicit-CN barotropic Helmholtz, KPP boundary-layer diagnosis, GM/Redi triad scheme, and Wright EOS hydrostatic-pressure iteration. We have no precedent in the production-OGCM world; the closest comparison is MITgcm/ECCO ([Heimbach et al. 2005], [Forget et al. 2015]), but MITgcm’s adjoint is generated by TAF (source transformation) rather than operator-overloading AD.
- Veros (Häfner et al. 2021) is the only other JAX-native ocean model we are aware of; we have a richer dycore (TRiSK MPAS, partial cells, full GM/Redi triads, implicit-CN barotropic) but are still climbing the validation ladder Veros has already passed.

Literature pointer

[Heimbach, Hill & Giering 2005], [Forget et al. 2015], [Häfner et al. 2021], [Ross et al. 2023] (online training of learned closures).

Known issues / caveats

- `bottom_level` (`OceanPartialCellCoordinate.bottom_level`) is integer-valued. AD is smooth in H_{bathy} only while `bottom_level` does not change — i.e., piecewise smooth with dis-

continuities at every reference-level interface (`docs/ocean_experiments/partial_cells_plan.md` “Differentiability Contract”).

- Gradient checkpointing through long barotropic substep loops is **not** auto-applied; for production-class gradient computation the user must wrap with `jax.checkpoint`.

21. What we run in production

The full ocean test matrix is the source of truth: `scripts/ocean_test_matrix/testcase.py:_build_test_matrix`. It generates 100+ cases spanning four grid families:

- **Rest-state preservation** (4 variants $\times$ 4 grids) — most fundamental PGF / metric / halo check.
- **Barotropic gravity wave** — resolution-matched (~ 384 – 446 km) on all grids.
- **Wind-driven regional gyres** (barotropic and baroclinic; cosine and $\sin^2$ wind profiles) — on regional grids.
- **Global barotropic wind-driven** — lat-lon and MPAS only (cubed-sphere excluded due to instability); 1-level and 10-level variants (10-level previously broken by issue #103, now fixed via Coriolis perturbation split).
- **Geostrophic adjustment** on all grids.
- **Phillips two-layer baroclinic** on all grids.
- **Inertia-gravity wave** (Bishnu et al. 2024 protocol) on all grids.
- **Lock exchange and overflow** (NEMO / Petersen et al. 2015 protocols) on cubed-sphere and lat-lon.
- **Stommel gyre tracer** (Hecht et al. 2000) on cubed-sphere, lat-lon, MPAS.
- **Eady baroclinic instability** on zonally periodic channel grids (lat-lon and MPAS); both eddy-permitting and parameterized (GM/Redi) variants.
- **Eady GM/Redi** with both `slope_scheme="centered"` and `slope_scheme="triads"` — pinpointing the cross-isopycnal residual.
- **ACC channel with Gaussian ridge** (Zhang et al. 2024-inspired) on lat-lon and MPAS channels (memory `project_acc_channel.md`).

Operational integrations

- **AMIP**: `scripts/run_amip.py`.
- **Global overturning**: lat-lon $\frac{1}{4}^\circ$ and MPAS ico-4 (memory `project_global_overturning.md`).

The matrix is auto-driven by `scripts/run_ocean_test_matrix.py` and produces side-by-side plots and diagnostics for all four grids. The `docs/CROSS_GRID_COMPARISON_REPORT.md` is the historical spotcheck.

22. Open scientific questions for Alistair

The following are the questions we would most value Alistair’s opinion on. Each is anchored in this document.

Q1 — Cubed-sphere ocean instability

The cubed-sphere C-D grid ocean has an exponential face-boundary instability (e-folding ~ 0.8 days on geostrophic adjustment; memory `project_cubesphere_ocean_instability.md`). MPAS and lat-lon are stable on the same tests. We attribute it to a compounding chain (mask interpolation at panel boundaries, C-D mismatch in PGF stencils, ocean's $\rho \rightarrow p \rightarrow \mathbf{u} \rightarrow \text{advect} \rightarrow \rho$ feedback loop). **Has Alistair seen anything analogous in any MITgcm cubed-sphere ocean experiment? What does MITgcm LLC do at the panel boundaries to avoid this?** (Our plan is a pure-C-grid cubed-sphere ocean modelled on LLC; we want to validate this direction.)

Q2 — Lat-lon barotropic mode noise (cosine filter)

We have grid-scale ± 5 cm/s noise in time-mean $\bar{V}$ on lat-lon C-grid that doesn't average to zero over 50,000 timesteps; amplifies via $\rho H f \approx 5 \times 10^5$ to ~ 0.1 Pa westward force on momentum-budget diagnostics. None of divergence damping, NEMO-style doubled boxcar, or ROMS power-law filter resolved it. Only **implicit-CN** structurally fixes it (memory `project_barotropic_noise_issue.md`; `docs/issues/barotropic_mode_noise.md`). **Is MOM6's predictor-corrector boxcar with full implicit-gravity-wave treatment the right answer here, or do you recommend something else? Should we make implicit-CN the default on lat-lon as well as MPAS?**

Q3 — MPAS+ETOPO bottom-trapped instability

Multi-year stability at MPAS+ETOPO+ico-4 within the current dycore architecture is **not** achievable. The damping toolkit (A_h , B_h , Smagorinsky, Leith, equatorial boost, barotropic-mode biharmonic, BBL drag, K_ζ -biharmonic) and higher resolution (ico-5) and all four PGF schemes (centered, adcroft, smc03, ahh08) have been exhausted (memory `project_mpas_etopo_instability.md` §8g–§8k). The §8f operator- elimination matrix isolated **PGF at partial cells** as the energy injector — but seed reduction (static $\rho_{\text{ref}}(z)$, AHH08) gives **no** improvement under the production stack because the day-30+ growth is amplification-dominated, not seed-dominated. **What does MOM6 do differently? Is AHH08 the right answer with a different amplification-side fix? Should we move to a generalised vertical coordinate?**

Q4 — GM/Redi triad on MPAS Voronoi

Triads (Griffies, Gnanadesikan, Pacanowski et al. 1998) are required for century-scale climate runs because centered Redi has a small but cumulative cross-isopycnal residual that compounds through pressure-velocity-advection feedback. We have triads on lat-lon C-grid (memory `project_gm_redi_latlon.md`) and centered on MPAS. **Has anyone implemented triads correctly on a Voronoi mesh? The 4-triad-per-face / 8-triad-per-w-face stencil construction needs careful geometric reasoning on the dual mesh.**

Q5 — Move to fully implicit barotropic everywhere?

Implicit-CN barotropic is validated on 5-yr MPAS ico-4 (memory `project_mpas_barotropic_noise.md`): σ_{grid} plateaus at 1.93×10^{-2} m/s vs explicit's growth to 9.93×10^{-2} . **Are there any contraindi-**

cations to making implicit-CN the default on lat-lon as well? Wall-clock comparable; differentiable; eliminates Coriolis null-mode by construction.

Q6 — Time filter design

We currently use a cosine (Hanning) time filter on the barotropic substep accumulators. None of MOM6 / MITgcm / NEMO / POP / MPAS-O / ROMS use a cosine filter ([Shchepetkin & McWilliams 2005] excepted, and only as a weighting); they use higher-order filters or fully-implicit gravity-wave treatment. **What’s the standard at GFDL?** Our staged-fix plan considers MOM6-style boxcar with predictor-corrector, ROMS-style power-law ($p = 2, q = 4, r = 0.284$), or fully implicit. The simplest move might be just to commit to implicit-CN globally.

Q7 — Vertical coordinate / ALE

We use $\mathbf{z}^*$ + **partial cells** end-to-end. We do not have an ALE remap step. Spurious diapycnal mixing is a known concern ([Griffies, Pacanowski & Hallberg 2000]; [Megann 2018]); SOM advection helps a lot, but it’s a horizontal-only mitigation. **Does Alistair recommend porting MOM6’s ALE Lagrangian + remap to legoESM? What does it buy us in terms of conservation, diapycnal mixing, scientific fidelity?** The differentiability question for ALE is open — **remap is piecewise and has discontinuities when layer ordering changes** ([White & Adcroft 2008]). We would value a sense of whether the AD complications are real or surmountable.

Q8 — Tracer transport in vanishing layers

Our flux-form tracer update is

$$T_{\text{new}} = \frac{h_{\text{old}}T - dt \cdot F}{\max(h_{\text{new}}, 10^{-10})}$$

with a hard floor on h_{new} and an `active_3d` post-mask that preserves pre-step values in below-seafloor cells. **What does MOM6 do here?** Specifically: how does the ALE remap handle a vanishing layer in flux-form transport without the divide-by-tiny-h amplifying numerical residuals into huge spurious tracer values? (We saw this exact failure mode in pre-PR-#231 on lat-lon and in “Audit 2” on MPAS, both fixed by the `active_3d` gate.)

Q9 — Bottom drag implementation

We have linear/quadratic/quadratic-with-floor, with optional BBL distribution (Killworth-Edwards 1999 / MOM6 `BBL_thick_min`), and explicit implicit-Euler treatment in the barotropic substep loop. Implicit-CN does **not** include drag in the Helmholtz operator (memory `project_mom6_slow_forcing.md`); it relies on F_{slow} . **Is this stable enough or do we need to follow MOM6’s design that wires drag into the barotropic Helmholtz operator directly?**

Q10 — Bonus: differentiability question

We achieve `jax.grad` through MPI halo exchange, implicit-CN PCG, KPP, GM/Redi triads, and Wright EOS. Is the production-OGCM community ready to consume *gradient* output for, e.g., parameter calibration, observational inversion, or learned closures, in the way ECCO consumes MITgcm adjoint output? ([Forget et al. 2015]; [Ross et al. 2023].) Or does the operator-overloading-AD output need a different kind of post-processing pipeline before it can be used at production scale?

Bibliography

Sources confirmed from the persona file’s reference list; legoESM file paths cited inline above.

- **Adcroft, A. & Campin, J.-M. (2004).** Rescaled height coordinates for accurate representation of free-surface flows in ocean circulation models. *Ocean Modelling*, 7, 269–284.
- **Adcroft, A., Hallberg, R. (2006).** On methods for solving the oceanic equations of motion in generalized vertical coordinates. *Ocean Modelling*, 11, 224–233.
- **Adcroft, A., Hallberg, R., Harrison, M. (2008).** A finite volume discretization of the pressure gradient force using analytic integration. *Ocean Modelling*, 22, 106–113.
- **Adcroft, A., Hill, C., Marshall, J. (1997).** Representation of topography by shaved cells in a height-coordinate ocean model. *Monthly Weather Review*, 125, 2293–2315.
- **Adcroft, A. & Marshall, J. (1998).** How slippery are piecewise-constant coastlines in numerical ocean models? *Tellus A*, 50, 95–108.
- **Adcroft, A., et al. (2019).** The GFDL global ocean and sea ice model OM4.0. *Journal of Advances in Modeling Earth Systems*, 11.
- **Arakawa, A. & Hsu, Y.-J. G. (1990).** Energy conserving and potential-entropy dissipating schemes for the shallow water equations. *Monthly Weather Review*, 118, 1960–1969.
- **Arakawa, A. & Lamb, V. R. (1977).** Computational design of the basic dynamical processes of the UCLA general circulation model. *Methods in Computational Physics*, 17, 173–265.
- **Arakawa, A. & Lamb, V. R. (1981).** A potential enstrophy and energy conserving scheme for the shallow water equations. *Monthly Weather Review*, 109, 18–36.
- **Bleck, R. (2002).** An oceanic general circulation model framed in hybrid isopycnic-Cartesian coordinates. *Ocean Modelling*, 4, 55–88.
- **Colella, P. & Woodward, P. R. (1984).** The piecewise parabolic method (PPM) for gas-dynamical simulations. *Journal of Computational Physics*, 54, 174–201.
- **Danabasoglu, G. & Marshall, J. (2007).** Effects of vertical variations of thickness diffusivity in an ocean general circulation model. *Ocean Modelling*, 18, 122–141.
- **Forget, G., Campin, J.-M., Heimbach, P., et al. (2015).** ECCO version 4: an integrated framework for non-linear inverse modeling and global ocean state estimation. *Geoscientific Model Development*, 8, 3071–3104.
- **Fox-Kemper, B., Ferrari, R., Hallberg, R. (2008).** Parameterization of mixed layer eddies. *Journal of Physical Oceanography*, 38, 1145–1165.
- **Fox-Kemper, B. & Menemenlis, D. (2008).** Can large eddy simulation techniques improve mesoscale rich ocean models? *Ocean Modeling in an Eddying Regime*, AGU Monograph 177, 319–337.

- **Gent, P. R. & McWilliams, J. C. (1990).** Isopycnal mixing in ocean circulation models. *Journal of Physical Oceanography*, 20, 150–155.
- **Griffies, S. M. (1998).** The Gent-McWilliams skew flux. *Journal of Physical Oceanography*, 28, 831–841.
- **Griffies, S. M. (2004).** *Fundamentals of Ocean Climate Models*. Princeton University Press.
- **Griffies, S. M. & Adcroft, A. (2008).** Formulating the equations of ocean models. In *Ocean Modeling in an Eddying Regime*, AGU Monograph 177, 281–317.
- **Griffies, S. M., Gnanadesikan, A., Pacanowski, R. C., Larichev, V. D., Dukowicz, J. K., Smith, R. D. (1998).** Isoneutral diffusion in a z-coordinate ocean model. *Journal of Physical Oceanography*, 28, 805–830.
- **Griffies, S. M. & Hallberg, R. (2000).** Biharmonic friction with a Smagorinsky-like viscosity for use in large-scale eddy-permitting ocean models. *Monthly Weather Review*, 128, 2935–2946.
- **Griffies, S. M., Pacanowski, R. C., Hallberg, R. W. (2000).** Spurious diapycnal mixing associated with advection in a z-coordinate ocean model. *Monthly Weather Review*, 128, 538–564.
- **Häfner, D., Jacobsen, R. L., Eden, C., Kristensen, M. R. B., Jansen, M., Visbeck, M. (2021).** Veros v0.1 — a fast and versatile ocean simulator in pure Python. *Geoscientific Model Development*, 14, 5343–5364.
- **Hallberg, R. (1997).** Stable split time stepping schemes for large-scale ocean modeling. *Journal of Computational Physics*, 135, 54–65.
- **Hallberg, R. & Adcroft, A. (2009).** Reconciling estimates of the free surface height in Lagrangian vertical coordinate ocean models with mode-split time stepping. *Ocean Modelling*, 29, 15–26.
- **Heimbach, P., Hill, C., Giering, R. (2005).** An efficient exact adjoint of the parallel MIT general circulation model, generated via automatic differentiation. *Future Generation Computer Systems*, 21, 1356–1371.
- **Hecht, M. W., et al. (2000).** Numerical experience with the POP code on the Cray T3E. *Ocean Modelling*, 2, 65–94. (Stommel gyre tracer set-up.)
- **Higdon, R. L. (2005).** A two-level time-stepping method for layered ocean circulation models. *Journal of Computational Physics*, 209, 459–502.
- **Hollingsworth, A., Kållberg, P., Renner, V. (1983).** An internal symmetric computational instability. *Quarterly Journal of the Royal Meteorological Society*, 109, 417–428.
- **Hundsdorfer, W., Koren, B., van Loon, M., Verwer, J. G. (1995).** A positive finite-difference advection scheme. *Journal of Computational Physics*, 117, 35–46. (Basis for MITgcm DST-3.)
- **Ilicak, M., Adcroft, A., Griffies, S. M., Hallberg, R. W. (2012).** Spurious dianeutral mixing and the role of momentum closure. *Ocean Modelling*, 45–46, 37–58. (Lock exchange, overflow, rest-state protocols.)
- **Jansen, M. F., Adcroft, A., Hallberg, R., Held, I. M. (2015).** Parameterization of eddy fluxes based on a mesoscale energy budget. *Ocean Modelling*, 92, 28–41.
- **Killworth, P. D., Webb, D. J., Stainforth, D., Paterson, S. M. (1991).** The development of a free-surface Bryan-Cox-Semtner ocean model. *Journal of Physical Oceanography*, 21, 1333–1348.
- **Killworth, P. D. & Edwards, N. R. (1999).** A turbulent bottom boundary layer code for use in numerical ocean models. *Journal of Physical Oceanography*, 29, 1221–1238.
- **Large, W. G., McWilliams, J. C., Doney, S. C. (1994).** Oceanic vertical mixing: a review and a model with a nonlocal boundary layer parameterization. *Reviews of Geophysics*,

32, 363–403.

- **Large, W. G. & Yeager, S. G. (2009).** The global climatology of an interannually varying air-sea flux data set. *Climate Dynamics*, 33, 341–364.
- **Leith, C. E. (1996).** Stochastic models of chaotic systems. *Physica D*, 98, 481–491.
- **Losch, M., Adcroft, A., Campin, J.-M. (2004).** How sensitive are coarse general circulation models to fundamental approximations in the equations of motion? *Journal of Physical Oceanography*, 34, 306–319.
- **Madec, G., et al. (2022).** NEMO Ocean Engine version 4.2. *Notes du Pôle de modélisation de l’IPSL*, No 27.
- **Marshall, D. P., Maddison, J. R., Berloff, P. S. (2012).** A framework for parameterizing eddy potential vorticity fluxes. *Journal of Physical Oceanography*, 42, 539–557.
- **Megann, A. (2018).** Estimating the numerical diapycnal mixing in an eddy-permitting ocean model. *Ocean Modelling*, 121, 19–33.
- **Mellor, G. L., Oey, L.-Y., Ezer, T. (1998).** Sigma coordinate pressure gradient errors and the seamount problem. *Journal of Atmospheric and Oceanic Technology*, 15, 1122–1131.
- **Pacanowski, R. C. & Gnanadesikan, A. (1998).** Transient response in a z-level ocean model that resolves topography with partial cells. *Monthly Weather Review*, 126, 3248–3270.
- **Pacanowski, R. C. & Philander, S. G. H. (1981).** Parameterization of vertical mixing in numerical models of tropical oceans. *Journal of Physical Oceanography*, 11, 1443–1451.
- **Paulson, C. A. & Simpson, J. J. (1977).** Irradiance measurements in the upper ocean. *Journal of Physical Oceanography*, 7, 952–956.
- **Petersen, M. R., et al. (2015).** Evaluation of the arbitrary Lagrangian–Eulerian vertical coordinate method in the MPAS-Ocean model. *Ocean Modelling*, 86, 93–113.
- **Prather, M. J. (1986).** Numerical advection by conservation of second-order moments. *Journal of Geophysical Research*, 91, 6671–6681.
- **Putman, W. M. & Lin, S.-J. (2007).** Finite-volume transport on various cubed-sphere grids. *Journal of Computational Physics*, 227, 55–78.
- **Redi, M. H. (1982).** Oceanic isopycnal mixing by coordinate rotation. *Journal of Physical Oceanography*, 12, 1154–1158.
- **Reichl, B. G. & Hallberg, R. (2018).** A simplified energetics based planetary boundary layer (ePBL) approach for ocean climate simulations. *Ocean Modelling*, 132, 112–129.
- **Rhines, P. B. (1969).** Slow oscillations in an ocean of varying depth. Part I. Abrupt topography. *Journal of Fluid Mechanics*, 37, 161–189.
- **Ringler, T. D., Thuburn, J., Klemp, J. B., Skamarock, W. C. (2010).** A unified approach to energy conservation and potential vorticity dynamics for arbitrarily-structured C-grids. *Journal of Computational Physics*, 229, 3065–3090.
- **Ringler, T. D., et al. (2013).** A multi-resolution approach to global ocean modeling. *Ocean Modelling*, 69, 211–232.
- **Ronchi, C., Iacono, R., Paolucci, P. S. (1996).** The “cubed sphere”: a new method for the solution of partial differential equations in spherical geometry. *Journal of Computational Physics*, 124, 93–114.
- **Roquet, F., Madec, G., McDougall, T. J., Barker, P. M. (2015).** Accurate polynomial expressions for the density and specific volume of seawater using the TEOS-10 standard. *Ocean Modelling*, 90, 29–43.
- **Ross, A., Li, Z., Perezhugin, P., Fernandez-Granda, C., Zanna, L. (2023).** Benchmarking of machine learning ocean subgrid parameterizations in an idealized model. *JAMES*, 15.
- **Sadourny, R. (1975).** The dynamics of finite-difference models of the shallow-water equa-

tions. *Journal of Atmospheric Sciences*, 32, 680–689.

- **Sadourny, R. & Basdevant, C. (1985).** Parameterization of subgrid-scale barotropic and baroclinic eddies in quasi-geostrophic models: anticipated potential vorticity method. *Journal of Atmospheric Sciences*, 42, 1353–1363.
- **Shchepetkin, A. F. & McWilliams, J. C. (2003).** A method for computing horizontal pressure-gradient force in an oceanic model with a non-aligned vertical coordinate. *Journal of Geophysical Research*, 108(C3), 3090.
- **Shchepetkin, A. F. & McWilliams, J. C. (2005).** The Regional Oceanic Modeling System (ROMS): a split-explicit, free-surface, topography-following-coordinate oceanic model. *Ocean Modelling*, 9, 347–404.
- **Silvestri, S., et al. (2024).** A new WENO-based momentum advection scheme for simulations of ocean mesoscale turbulence. *Journal of Advances in Modeling Earth Systems*, 16.
- **Simmons, H. L., Jayne, S. R., St. Laurent, L. C., Schmittner, A. (2004).** Tidally driven mixing in a numerical model of the ocean general circulation. *Ocean Modelling*, 6, 245–263.
- **Smagorinsky, J. (1963).** General circulation experiments with the primitive equations. *Monthly Weather Review*, 91, 99–164.
- **Thuburn, J., Ringler, T. D., Skamarock, W. C., Klemp, J. B. (2009).** Numerical representation of geostrophic modes on arbitrarily structured C-grids. *Journal of Computational Physics*, 228, 8321–8335.
- **Van Roekel, L. P., et al. (2018).** The KPP boundary layer scheme for the ocean: revisiting its formulation and benchmarking one-dimensional simulations. *JAMES*, 10.
- **Visbeck, M., Marshall, J., Haine, T., Spall, M. (1997).** Specification of eddy transfer coefficients in coarse-resolution ocean circulation models. *Journal of Physical Oceanography*, 27, 381–402.
- **White, L. & Adcroft, A. (2008).** A high-order finite volume remapping scheme for nonuniform grids: the piecewise quartic method (PQM). *Journal of Computational Physics*, 227, 7394–7422.
- **Wright, D. G. (1997).** An equation of state for use in ocean models: Eckart’s formula revisited. *Journal of Atmospheric and Oceanic Technology*, 14, 735–740.
- **Zalesak, S. T. (1979).** Fully multidimensional flux-corrected transport algorithms for fluids. *Journal of Computational Physics*, 31, 335–362.